

GUÍA DE INSTALACIÓN

Sistema Comunitario de Gestión y Control del Agua

Repositorio: proyecto-analisis-de-sistemas

URL: <https://github.com/javier6441112/proyecto-analisis-de-sistemas/tree/main>

Tecnologías principales: Python, Flask, PostgreSQL, HTML, CSS, JavaScript y Docker

Tipo de documento: Guía de instalación y configuración del entorno de desarrollo

Contenido

1. Introducción
2. Requisitos previos del sistema
3. Herramientas necesarias
4. Clonación del repositorio
5. Instalación con Docker
6. Instalación local sin Docker
7. Configuración de variables de entorno
8. Inicialización de la base de datos
9. Ejecución del proyecto
10. Despliegue local o en servidor
11. Configuración del entorno de desarrollo
12. Comandos útiles
13. Verificación del funcionamiento
14. Problemas comunes y soluciones
15. Consideraciones de seguridad
16. Resumen de instalación rápida

1. Introducción

La presente guía describe el proceso de instalación, configuración y ejecución del proyecto **Sistema Comunitario de Gestión y Control del Agua**, desarrollado como una aplicación web para administrar información relacionada con el almacenamiento, consumo, distribución, pagos, mantenimiento y notificaciones dentro de una comunidad.

El sistema está construido utilizando **Python con Flask** para el backend, **PostgreSQL** como gestor de base de datos y tecnologías web estándar como **HTML, CSS y JavaScript** para la interfaz de usuario.

Esta guía contempla dos formas de instalación:

- **Instalación con Docker:** recomendada para levantar todo el entorno de forma rápida.
- **Instalación local sin Docker:** recomendada cuando se desea trabajar directamente con Python y PostgreSQL instalados en la computadora.

2. Requisitos previos del sistema

Antes de instalar el proyecto, se debe contar con un sistema operativo compatible y las herramientas necesarias para ejecutar una aplicación web basada en Python y PostgreSQL.

2.1 Sistema operativo recomendado

El proyecto puede ejecutarse en los siguientes sistemas operativos:

- Windows 10 o Windows 11
- Linux Ubuntu, Debian o distribuciones similares
- macOS

Recomendación: Para fines académicos o de desarrollo local se recomienda utilizar Windows con Visual Studio Code o Linux Ubuntu.

2.2 Recursos mínimos sugeridos

Recurso	Recomendación mínima
Memoria RAM	4 GB como mínimo, 8 GB recomendado
Procesador	Procesador de 2 núcleos o superior
Almacenamiento	2 GB libres como mínimo
Conexión a internet	Necesaria para clonar el repositorio e instalar dependencias

3. Herramientas necesarias

3.1 Herramientas para instalación con Docker

Para ejecutar el proyecto usando Docker se necesita instalar:

- Git
- Docker Desktop
- Docker Compose
- Visual Studio Code, opcional pero recomendado

Esta opción es la más recomendada porque permite levantar tanto la aplicación Flask como la base de datos PostgreSQL mediante el archivo `docker-compose.yml`.

3.2 Herramientas para instalación local sin Docker

Para ejecutar el proyecto directamente en la computadora se necesita:

- Python 3.12 o versión compatible
- PostgreSQL instalado localmente
- Git

- pip
- Entorno virtual de Python
- Visual Studio Code

4. Clonación del repositorio

El primer paso es descargar el código fuente del proyecto desde GitHub. Para ello se debe abrir una terminal o la terminal integrada de Visual Studio Code y ejecutar:

```
git clone https://github.com/javier6441112/proyecto-analisis-de-sistemas.git
```

Luego se debe ingresar a la carpeta del proyecto:

```
cd proyecto-analisis-de-sistemas
```

Después de este paso, el código fuente del sistema estará disponible localmente para su instalación, configuración y ejecución.

5. Instalación con Docker

La instalación con Docker permite ejecutar la aplicación y la base de datos dentro de contenedores. Esto evita instalar PostgreSQL y varias configuraciones directamente en el sistema operativo.

5.1 Verificar archivos principales

En la raíz del proyecto se deben encontrar archivos similares a los siguientes:

```
Dockerfile
docker-compose.yml
requirements.txt
run.py
.env.example
app/
```

El archivo `Dockerfile` define cómo se construye la imagen de la aplicación Flask. El archivo `docker-compose.yml` permite levantar los servicios necesarios del sistema.

5.2 Construir y levantar los contenedores

Desde la raíz del proyecto ejecutar:

```
docker compose up -d --build
```

Este comando realiza las siguientes acciones:

- Construye la imagen Docker de la aplicación.
- Levanta el contenedor de PostgreSQL.
- Levanta el contenedor de la aplicación Flask.
- Conecta ambos servicios mediante la red interna de Docker Compose.
- Expone la aplicación en el puerto 5000.

5.3 Verificar contenedores activos

Para confirmar que los contenedores se encuentran en ejecución, usar:

```
docker ps
```

También se puede utilizar:

```
docker compose ps
```

Deben aparecer al menos dos servicios activos: uno correspondiente a la aplicación web y otro correspondiente a PostgreSQL.

6. Instalación local sin Docker

Esta alternativa se utiliza cuando se desea ejecutar el proyecto directamente en la computadora, usando Python y PostgreSQL instalados manualmente.

6.1 Crear entorno virtual

En la raíz del proyecto, crear un entorno virtual de Python.

En Windows

```
python -m venv .venv
```

Luego activar el entorno virtual:

```
.venv\Scripts\activate
```

En Linux o macOS

```
python3 -m venv .venv
```

Luego activar el entorno virtual:

```
source .venv/bin/activate
```

6.2 Instalar dependencias

Con el entorno virtual activado, instalar las dependencias del proyecto:

```
pip install -r requirements.txt
```

Este comando instala las librerías necesarias para ejecutar la aplicación, como Flask, SQLAlchemy, herramientas de migración, conexión con PostgreSQL, manejo de variables de entorno y autenticación mediante JWT.

7. Configuración de variables de entorno

Las variables de entorno permiten configurar valores sensibles o dependientes del entorno, como la conexión a la base de datos y las claves secretas del sistema.

7.1 Crear archivo .env

El repositorio incluye un archivo de ejemplo llamado `.env.example`. Se debe copiar este archivo y renombrarlo como `.env`.

En Windows

```
copy .env.example .env
```

En Linux o macOS

```
cp .env.example .env
```

7.2 Variables principales

El archivo `.env` debe contener una configuración similar a la siguiente:

```
FLASK_ENV=development
SECRET_KEY=cambia-esta-clave
JWT_SECRET=cambia-esta-clave-jwt
DATABASE_URL=postgresql+psycopg2://agua_user:agua_pass@localhost:5432/agua_db
```

Variable	Descripción
FLASK_ENV	Define el entorno de ejecución de Flask. Para desarrollo se utiliza <code>development</code> .
SECRET_KEY	Clave interna utilizada por Flask para seguridad general de la aplicación.
JWT_SECRET	Clave utilizada para firmar y validar tokens JWT.
DATABASE_URL	Cadena de conexión hacia la base de datos PostgreSQL.

7.3 Configuración de DATABASE_URL para Docker

Si el proyecto se ejecuta con Docker Compose, el host de la base de datos debe ser el nombre del servicio definido para PostgreSQL. Normalmente se utiliza `db`.

```
DATABASE_URL=postgresql+psycpg2://agua_user:agua_pass@db:5432/agua_db
```

Importante: Dentro de Docker, no se debe usar `localhost` para conectar la aplicación con PostgreSQL, porque cada servicio se ejecuta en un contenedor separado. Por eso se utiliza el nombre del servicio, por ejemplo `db`.

7.4 Configuración de `DATABASE_URL` para ejecución local

Si el proyecto se ejecuta sin Docker, la base de datos se encuentra en la misma computadora. En ese caso sí se utiliza `localhost`.

```
DATABASE_URL=postgresql+psycpg2://agua_user:agua_pass@localhost:5432/agua_db
```

8. Configuración de PostgreSQL local

Este paso solo aplica si se ejecutará el proyecto sin Docker. Si se utiliza Docker, PostgreSQL se crea automáticamente por medio del archivo `docker-compose.yml`.

8.1 Crear base de datos y usuario

Ingresar a PostgreSQL usando una herramienta como pgAdmin, DBeaver, la terminal de PostgreSQL o cualquier cliente compatible. Luego ejecutar:

```
CREATE DATABASE agua_db;  
  
CREATE USER agua_user WITH PASSWORD 'agua_pass';  
  
GRANT ALL PRIVILEGES ON DATABASE agua_db TO agua_user;
```

Estos datos deben coincidir con la cadena de conexión definida en el archivo `.env`.

9. Inicialización de la base de datos

Una vez configurada la conexión, se deben crear las tablas e insertar los datos iniciales del sistema.

9.1 Inicializar base de datos con Docker

Ejecutar el siguiente comando:

```
docker compose exec web flask init-db
```

Este comando ejecuta el proceso de inicialización dentro del contenedor de la aplicación web.

9.2 Inicializar base de datos en instalación local

Si el proyecto se ejecuta directamente con Python, usar:

```
flask --app run.py init-db
```

Este proceso crea las tablas necesarias y registra datos iniciales para poder probar el sistema.

10. Ejecución del proyecto

10.1 Ejecutar con Docker

Si los contenedores ya fueron levantados, la aplicación debería estar disponible automáticamente.

```
docker compose up -d
```

Abrir en el navegador:

```
http://localhost:5000
```

10.2 Ejecutar localmente sin Docker

Con el entorno virtual activado y la base de datos configurada, ejecutar:

```
python run.py
```

Abrir en el navegador:

```
http://localhost:5000
```

11. Despliegue local o en servidor

Para desplegar el sistema en una máquina servidor, se recomienda utilizar Docker, ya que permite instalar y ejecutar la aplicación de forma más ordenada.

11.1 Clonar el repositorio en el servidor

```
git clone https://github.com/javier6441112/proyecto-analisis-de-sistemas.git  
cd proyecto-analisis-de-sistemas
```

11.2 Levantar servicios

```
docker compose up -d --build
```

11.3 Inicializar base de datos

```
docker compose exec web flask init-db
```

11.4 Acceso desde otra computadora de la red

Para acceder desde otra computadora, primero se debe obtener la dirección IP del servidor.

En Windows

```
ipconfig
```

En Linux

```
ip addr
```

Luego, desde otra computadora conectada a la misma red, abrir:

```
http://IP_DEL_SERVIDOR:5000
```

Ejemplo:

```
http://192.168.0.50:5000
```

Nota: Si no se puede acceder desde otra computadora, se debe revisar que el firewall del servidor permita conexiones al puerto 5000.

12. Guía de configuración del entorno de desarrollo

12.1 Abrir proyecto en Visual Studio Code

Desde la terminal, dentro de la carpeta del proyecto, ejecutar:

```
code .
```

También se puede abrir Visual Studio Code y seleccionar la opción:

```
File > Open Folder
```

Luego seleccionar la carpeta del proyecto clonado.

12.2 Extensiones recomendadas

Para trabajar de forma más cómoda con el proyecto, se recomienda instalar las siguientes extensiones:

- Python
- Pylance
- Docker
- PostgreSQL
- GitLens

- Thunder Client o REST Client para probar endpoints

12.3 Estructura general del proyecto

La estructura del proyecto puede organizarse de forma similar a la siguiente:

```
proyecto-analisis-de-sistemas/  
├── app/  
│   ├── routes/  
│   │   ├── api.py  
│   │   └── auth.py  
│   ├── static/  
│   │   ├── css/  
│   │   └── js/  
│   ├── templates/  
│   │   └── index.html  
│   ├── extensions.py  
│   ├── models.py  
│   └── __init__.py  
├── Dockerfile  
├── docker-compose.yml  
├── requirements.txt  
├── run.py  
├── schema.sql  
├── openapi.yml  
└── .env.example
```

12.4 Descripción de carpetas y archivos principales

Archivo o carpeta	Descripción
app/	Contiene el código principal de la aplicación Flask.
app/routes/	Contiene las rutas de autenticación y las rutas API del sistema.
app/static/	Contiene archivos estáticos como CSS y JavaScript.
app/templates/	Contiene las plantillas HTML renderizadas por Flask.

Archivo o carpeta	Descripción
<code>models.py</code>	Define los modelos de datos utilizados por SQLAlchemy.
<code>extensions.py</code>	Contiene extensiones compartidas de Flask, como la conexión con la base de datos.
<code>run.py</code>	Archivo principal para ejecutar la aplicación.
<code>requirements.txt</code>	Lista de dependencias de Python necesarias para el proyecto.
<code>Dockerfile</code>	Define la imagen Docker de la aplicación.
<code>docker-compose.yml</code>	Define los servicios de la aplicación y la base de datos.
<code>.env.example</code>	Archivo de ejemplo para configurar variables de entorno.

13. Comandos útiles para desarrollo

13.1 Comandos Docker

Acción	Comando
Construir y levantar contenedores	<code>docker compose up -d --build</code>
Levantar servicios ya construidos	<code>docker compose up -d</code>
Ver contenedores activos	<code>docker ps</code>
Ver servicios del proyecto	<code>docker compose ps</code>
Ver logs de la aplicación	<code>docker compose logs -f web</code>
Ver logs de PostgreSQL	<code>docker compose logs -f db</code>
Apagar servicios	<code>docker compose down</code>
Apagar servicios y borrar volumen	<code>docker compose down -v</code>
Inicializar base de datos	<code>docker compose exec web flask init-db</code>

13.2 Comandos para ejecución local

Acción	Comando
Crear entorno virtual	<code>python -m venv .venv</code>
Activar entorno virtual en Windows	<code>.venv\Scripts\activate</code>
Activar entorno virtual en Linux/macOS	<code>source .venv/bin/activate</code>
Instalar dependencias	<code>pip install -r requirements.txt</code>
Inicializar base de datos	<code>flask --app run.py init-db</code>
Ejecutar aplicación	<code>python run.py</code>

14. Verificación del funcionamiento

Después de instalar y ejecutar el proyecto, se debe verificar que el sistema funcione correctamente.

14.1 Acceso a la aplicación

Abrir el navegador y entrar a:

```
http://localhost:5000
```

14.2 Usuario de prueba

Si la base de datos fue inicializada correctamente, se puede utilizar el usuario administrador de prueba:

Campo	Valor
DPI	1234567890101
Contraseña	Admin123*

14.3 Módulos a verificar

Una vez dentro del sistema, se recomienda revisar los siguientes módulos:

- Dashboard principal
- Agua almacenada
- Registro de viviendas
- Registro de residentes
- Consumo mensual de agua
- Pagos por servicio
- Distribución semanal
- Órdenes de mantenimiento
- Notificaciones

14.4 Prueba del endpoint del sensor

El sistema puede incluir un endpoint para registrar lecturas de agua enviadas por un sensor. Para probarlo se puede ejecutar:

```
curl -X POST http://localhost:5000/api/sensor \
  -H "Content-Type: application/json" \
  -d '{"liters": 9500, "observation": "Lectura enviada por sensor"}'
```

Si la petición se procesa correctamente, el sistema debe registrar la lectura en la base de datos.

15. Problemas comunes y soluciones

15.1 Error: no se puede conectar a PostgreSQL

Este error puede ocurrir cuando la variable `DATABASE_URL` está mal configurada o cuando el servicio de PostgreSQL no está activo.

Solución si se usa Docker

Verificar que los contenedores estén activos:

```
docker ps
```

Confirmar que el host de la base de datos sea `db`:

```
DATABASE_URL=postgresql+psycopg2://agua_user:agua_pass@db:5432/agua_db
```

Solución si se usa instalación local

Confirmar que PostgreSQL esté ejecutándose y que la conexión use `localhost`:

```
DATABASE_URL=postgresql+psycopg2://agua_user:agua_pass@localhost:5432/agua_db
```

15.2 Error: el puerto 5000 está ocupado

Si otro proceso está utilizando el puerto 5000, se puede cambiar el puerto expuesto en Docker Compose.

```
ports:
  - "5001:5000"
```

Luego se debe acceder desde:

```
http://localhost:5001
```

15.3 Error: las tablas no existen

Esto indica que la base de datos no fue inicializada correctamente.

Con Docker

```
docker compose exec web flask init-db
```

Sin Docker

```
flask --app run.py init-db
```

15.4 Error: dependencia no instalada

Si aparece un error indicando que falta una librería de Python, se deben instalar nuevamente las dependencias:

```
pip install -r requirements.txt
```

15.5 Error: Docker no reconoce el comando compose

Si el comando `docker compose` no funciona, verificar que Docker Desktop esté instalado y actualizado. En versiones antiguas puede utilizarse:

```
docker-compose up -d --build
```

16. Consideraciones de seguridad

Para un entorno académico, las claves de ejemplo pueden ser suficientes. Sin embargo, en un entorno real se deben aplicar medidas de seguridad adicionales.

- No subir el archivo `.env` al repositorio.
- Cambiar `SECRET_KEY` por una clave segura.
- Cambiar `JWT_SECRET` por una clave segura.
- Usar contraseñas fuertes para la base de datos.
- Configurar HTTPS en producción.
- Restringir el acceso directo a la base de datos.
- Realizar respaldos periódicos de PostgreSQL.
- No exponer credenciales dentro del código fuente.
- Evitar guardar información sensible en el navegador si el sistema se lleva a producción.

Advertencia: Las credenciales de prueba deben utilizarse únicamente para desarrollo o demostración. En producción deben reemplazarse por credenciales seguras.

17. Resumen de instalación rápida

17.1 Instalación rápida con Docker

```
git clone https://github.com/javier6441112/proyecto-analisis-de-sistemas.git
cd proyecto-analisis-de-sistemas
docker compose up -d --build
docker compose exec web flask init-db
```

Abrir en el navegador:

<http://localhost:5000>

17.2 Instalación rápida sin Docker

```
git clone https://github.com/javier6441112/proyecto-analisis-de-sistemas.git
cd proyecto-analisis-de-sistemas
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
flask --app run.py init-db
python run.py
```

Abrir en el navegador:

<http://localhost:5000>

Resultado esperado: Al finalizar estos pasos, el sistema debe estar ejecutándose correctamente y disponible desde el navegador en el puerto configurado.

18. Conclusión

Esta guía permite instalar, configurar y ejecutar el Sistema Comunitario de Gestión y Control del Agua tanto en un entorno local como en un entorno basado en Docker. La opción con Docker resulta más práctica para despliegues rápidos, ya que automatiza la creación de la base de datos y el levantamiento de servicios. La instalación local, por otro lado, es útil para desarrolladores que desean modificar y probar el código directamente desde su entorno de trabajo.

Con la configuración adecuada de las variables de entorno, la inicialización correcta de la base de datos y la ejecución del servidor Flask, el sistema queda listo para ser utilizado, probado o extendido con nuevas funcionalidades.